

Technical Report

BIG DATA ANALYTICS FINAL PROJECT: Distributed Multi-Model Analytics for E-commerce Data

1. Introduction

This project is about an analysis performed on an e-commerce platform to identify customer behavior and performance trends. Three different databases were used to perform these analytics as a method to compare the processing efficiency according to the kind of data stored and the query pattern. These metrics include the need for flexibility of documents, fast time-based lookups, and processing of distributed large-scale data.

The dataset used consists of records for the activities performed on the platform for 90 days. The dataset includes records of 10,000 users registered to the platform, 5,000 products available for transactions, 25 Categories in which each product on the platform belongs to, 500,000 transactions performed by the users on the platform, and 2,000,000 sessions that were recorded every time a user navigates through the platform.

2. System Architecture

After running the `data_generator.py` file shared from the project requirements, I was provided with 24 raw JSON files that store the users, products, categories, transactions, and 20 session files because of the huge size of data stored. These JSON files were loaded into MongoDB and HBase using their respective commands to unpack the data and store it. Several queries were performed in both databases to identify different patterns in the data, like top products by revenue, user segmentation by purchasing frequency, and getting all sessions for a specific user. Spark processing engine for distributed data was used as well to execute complex data workflows, like conducting a cohort analysis of user purchasing patterns by analyzing their spending over subsequent months, grouped by registration month cohorts. The results generated from these processes were later visualized using Python visualization libraries, namely matplotlib and seaborn, as per project requirements.

MongoDB was used because of its schema flexibility, which allowed me to process and store embedded documents that were present in our dataset, like items purchased per transaction, efficiently. HBase was also helpful for storing data in a column-based format, which allowed me to create row-key-based identifiers that enabled us to retrieve a user's complete session history without going through the whole table, and HBase provided us with the ability to create column families that allowed us to store fields with related information. Spark, on the other hand, allowed me to process large volume data processes like table joins used to identify user patterns according to their transactions, product affinity, allowing us to identify products frequently bought or viewed together, and combining browsing and purchase data into a single integrated analysis. Python libraries for visualizations are faster and don't require additional tools for generating visualizations.

3. MongoDB Design & Implementation

During the MongoDB design, I had five different 5 collections created after loading the different files generated after running the `dataset_generator.py` file.

- **Users collection:** The users' collection was created to store the details of the users registered on the platform. Registered data includes user identification, embedded document containing geographical data, date of registration, and a record of the last time the user was active on the platform.
- **Categories collection:** Every product registered on the platform is classified into a specific category. While loading the data into the MongoDB, I created a collection that is going to store these specific categories and an embedded array document containing subcategories found in each category alongside the category identification and the category name.
- **Product collection:** A collection containing product details was also created to store product information, such as the product identification, product name, category, subcategory, base price of the product, current stock available on the platform, creation date, and an embedded document containing the price history with its dates.
- **Sessions collection:** Every time a user logs to the platform, details of each activity performed and their occurrences were recorded which led to the creation of the session collection to store details like the session identification, the user logged in identification, the start of the session as well as its end, session duration in seconds, and embedded documents like the geographical data, the device used, the list of products viewed, the pages viewed during the session, the contents of the cart, conversion status as well as the referrer.
- **Transactions collection:** This collection stores all transactions performed by the users and their details, like the transaction identification, the session identification number, the user that performed the transaction, the date and time of the transaction, an embedded document of items purchased, details like the product identification, quantity purchased, unit price, and the subtotal, payment method, grand total, and the transaction status.

The presence of embedded documents was necessary due to the nature of the data being stored, having multiple values, but also being part of the collection as a document for that collection. After creating these collections, 3 aggregation pipelines were performed:

- **Top 10 products by revenue:** By reading all the items purchased from the transactions collection, it enabled me to identify the products purchased throughout the time. I grouped them individually and added their subtotals to give me the total revenue per product. I sorted them from highest to lowest revenue share and finally limited the output to only the first 10 products along with their revenues. Prod_00154 was the top result with \$343,418.70 revenues.
- **User segmentation by purchasing frequency:** After examining the data distribution of transactions per user, which allowed me to split the frequency of transactions per user, from the transaction collection, I grouped the users per frequency group, which showed 2,174 low-frequency users, 5,252 medium-frequency users, and 2,574 high-frequency users.
- **Total revenue by category:** Using transactions, products, and categories collections, I was able to retrieve the items purchased with the total of subtotal for each and identified the product name and category, then sorted them by total revenue to get the top categories. cat_005 with category name Cameron-Parsons came top with \$22,221,952.21 revenue from 40,189 orders.

4. HBase Design & Implementation

My HBase design consist of two tables named user_sessions and product_metrics. These tables were storing a composite row key and both have two column families. The user_sessions table stores a row key composed of a session's user identification and session's start time and three column families one for session info that holds the session id, session duration alongside device details used in the session like the device type, OS, browser. And finally an activity column family was created to store the viewed products and page counts. The product_metrics stores row key composed of the product id and the date followed by two column families namely product info and metrics to store revenue and number of order per product.

Since HBase is used for processing the analysis of huge amounts of data, the composite row key was used to combine the user ID and product ID with so that all of a user's or product's data sits together which allowed making lookups fast. A concept of column families was also used to group related data together so that the system doesn't have to skip loading the data I don't need while processing queries. Both these concepts helped in storing large amounts of data such as sessions data and products alongside them in order to easily identify related data and therefore make querying through the data fast and efficient.

This highlights the comparative justification of the use of both MongoDB and HBase in this system.

Criteria	MongoDB	HBase
Data Model	Document (Nested flexible)	Wide-column (sparse, flat)
Best for	Transactions, products, categories	Time-series session data
Query style	Aggregation pipelines	Row key scans, prefix scans
Schema flexibility	High flexibility allowing several variable fields per document	Moderate because of fixed column families
Scalability	Horizontal via sharding	Horizontal via region servers

Using the user sessions table, I was able to load 100,000 session data and 350,000 product purchased into the HBase database which allowed me to perform different queries:

- **All sessions for a specific user:** using the user sessions table, I managed to identify all sessions that user with ID user_000042 has logged into at different time periods which showed that this user has had 11 sessions on the platform
- **Daily Performance for a specific product:** by looking up into product metrics table, I was able to identify prod_00154 daily performance using the date it was registered in a transaction. The lookup showed that the total daily records found were 85 with a total of 578 units sold and a total revenue of \$343,418.70. However, to perform this analysis for each product, I first aggregated units sold per day and amount of revenue it generated that day, and how many separate orders included it.

5. Apache Spark Processing

During the apache spark processing, I encountered a hardware limitation while processing the 2 million worth session data because my personal computer could not accommodate that much processing, I instead worked with a sample of the session data. Spark driver required more memory than the 8GB memory that my personal computer has. And giving more than 4GB of memory to spark processing was freezing my computer there for I used the sample data that can be processed by the 4GB memory that my computer could provide. However other files like transactions and products were successfully loaded and processed.

During the data cleaning process, I found that almost half of the transactions performed did not have a session associated to it. This was an expected behavior because generation happened in two different codes. One that generated transaction from the session while the other randomly generated them to complete the 500,000 transactions which led to these transactions not having a session linked to it.

Using Apache spark I was able to perform different analysis on the data to identify patterns and trends:

- **Product affinity analysis:** This analysis used the transactions files to analyze how a product was purchased with the other in the same order. After aggregation of products bought together, the analysis showed that products bought together the most we bought 5 times which were prod_02483 and prod_02898.
- **Cohort analysis:** The analysis intended to identify the users that registered on the platform in the same month (cohort) and then tracking how their spending changed over time. The analysis showed 7 different monthly cohorts where cohort month 2026-01 came on top with 1,758 registrations. The analysis showed that the average spending is evenly distributed between \$920.00 - \$948.00 regardless of which cohort/month. This is a valid finding however it does not reflect the actual customer behaviors because the data was randomly generated, so no significant cohort effect was found in the dataset.
- **Spark SQL:** Using this spark SQL, I performed analysis on the payment methods for every transaction that was performed on the platform which showed that PayPal was mostly used as payment method where it was used 125,185 times followed with slight difference by credit card used 125,177. However, although these payment methods were used highly they did not produce much revenues on the platform because the highest revenue was generated by bank transfer with an average of \$1104.42 followed by gift cards with \$1102.90 average revenues. Additionally, using the Spark SQL Join query I was able to break down the revenues by country. By sorting them by total revenue the analysis showed that country with country code LA is the country with the highest revenue with \$3.15M total revenue followed by CZ with \$3.14M.

6. Integration Analysis

From the sessions recorded on the platform and the completed transactions I was able to answer a question that said: Do products that users frequently look at together end up being bought together, or does browsing interest fail to turn into actual sales?

To answer this question, I analyzed the products which were viewed together in the same browsing session and also analyzed the products which were brought together in the same order and processed

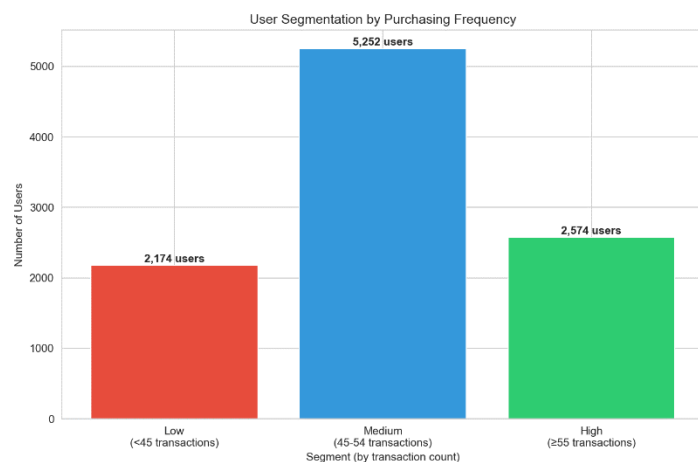
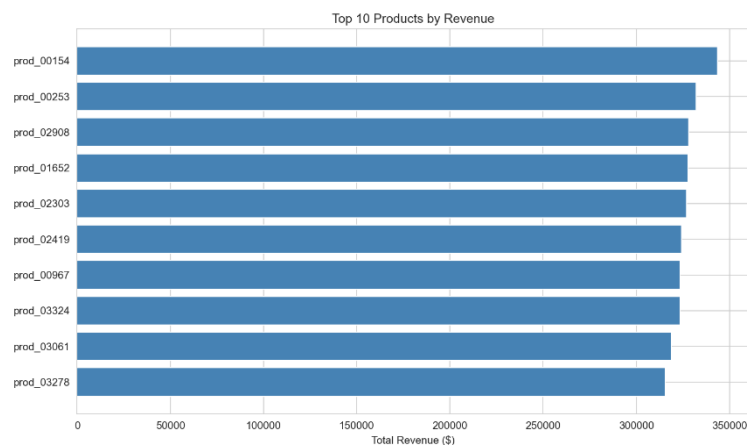
them into spark and compared them side by side. I started by identifying the possible pair of products that belong to a session or transaction containing more than one product, then counted how many times the pair occurred.

The amount of items viewed together and those bought together were then joined to identify the pair of products that were both viewed together then bought together.

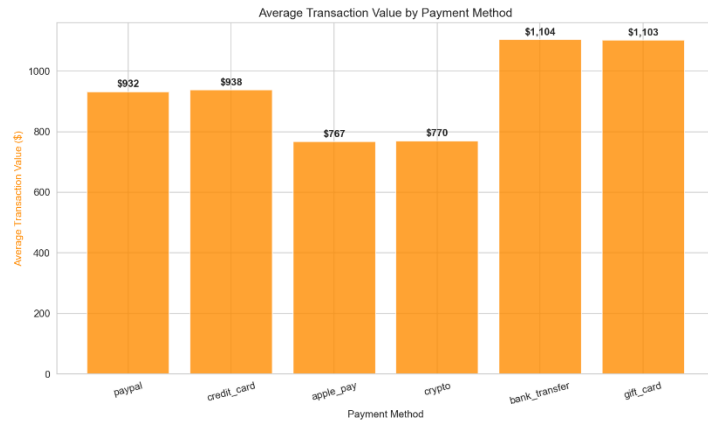
This analysis used a sample of 100,000 sessions and the full set of 500,000 transactions which produced a total of 570,929 pair of products. The analysis showed that when a user view different products at the same time, they end up buying them together as well. Pairs viewed together only once converted into a joint purchase about 7.5% of the time, while pairs viewed together three times converted about 12.5% of the time which is nearly double. This shows that repeated co-viewing is a meaningful signal of real purchase intent, not just random browsing.

7. Visualizations

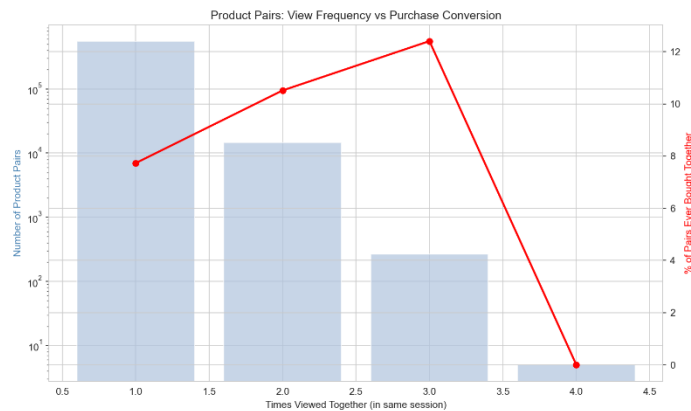
This visualization shows how the products provided revenue



visualization 2 shows how the users are segmented according to their purchasing frequency



Visualization 3 shows the average transaction value according their payment method



Visualization 4 shows how product pairs are compared to their view frequency and how often they are bought.

8. Limitations & Future Work

This project faced several key limitations. Limited hardware, specifically 8GB of RAM, meant that analysis had to be conducted on smaller data samples rather than the full dataset. Additionally, using synthetic data rather than real user data yielded patterns that were less realistic and more uniform, making the findings more illustrative than practical. There were also duplicate session IDs due to how the dataset was generated, but these were addressed during data processing. For future work, access to more powerful computing resources would allow for analysis of the complete dataset and eliminate the need for sampling. Incorporating advanced analytics, such as association rule mining, could provide deeper and more actionable insights. Expanding data tracking to cover longer time periods would also enable more comprehensive time-series analysis of product performance.